From Sparse Selection to Physical Savings: A Unified Runtime and Measurement Contract for Progressive Retrieval Attention

PRA Research Program

August 2026

Abstract

Progressive Retrieval Attention (PRA) lets a query select a bounded subset of URI-addressed, model-native key/value memory. Earlier work established the model interface, external-memory lifecycle, typed capability records, and compact typed-result backing, but those parts did not yet form one product surface, and logical sparsity did not by itself establish lower latency or accelerator memory. We introduce a unified runtime that preserves the existing Hugging Face API while adding an engine-neutral four-axis execution policy, request-owned selection plans, versioned systems configuration, authenticated sessions, lazy tool and skill disclosure, independently authorized tool execution, session-scoped result compaction, durable task-aware agent state, selective replay, deduplicated K/V interval planning, physical byte accounting, bounded caching, profiling, and a scheduler-unaware serving handoff. A standalone gateway now separates agent semantics, PRA mediation, and engine-native state through a JSON logical-reference contract and explicit capability negotiation. In a five-seed tiny-HF mechanism profile, token-shared selection required 2.8 routing operations per request, versus 8.4 for token-per-layer, a 3.0× reduction with three active layers. On a CUDA 12.6 host with an NVIDIA GTX 950M, gathering 512 of 8,192 native-K/V tokens takes a median 0.253 ms at batch one, and packing eight selected intervals takes 0.415 ms. The same packed payload costs 0.424 ms when HBM-resident, 1.592 ms from pinned CPU memory, and 2.157 ms from ordinary CPU memory. Thus, off-device transfer is 3.75–5.08 times the HBM-resident packing cost on this host. Compilation is a negative capability gate: the CPU toolchain lacks an available compiler and the GPU is below the Triton capability floor. No optional serving engine is installed. The result is therefore a tested portable eager baseline and reproducible measurement contract, not a fused-kernel or serving-engine speed claim.

1 Introduction

PRA changes what a model must keep logically available. A long-lived resource may remain addressable through a compact routing representation while only a small selected part of its native K/V participates in one attention operation. That distinction is algorithmically useful, but it leaves a systems question unanswered. A selected set can be small while the runtime still stores every detail tensor in HBM, launches many tiny gathers, copies the same bytes repeatedly, constructs temporary concatenations, or serializes requests around Python control flow. In that case sparsity exists in a trace but not in the resource bill.

This paper asks how to turn PRA into a portable inference primitive before a serving engine becomes semantically aware of PRA memory. The immediate task is not to design a new scheduler. It is to define an execution boundary that can be measured, optimized, and later replaced without changing the meaning of a selected reference.

Three previously separate interfaces must meet at that boundary. The Paper 2 SDK supplies `PRAForCausalLM`, model-family injection, routing, and generation. Paper 4 supplies authenticated

resolvers and the cold, warm, and hot external-memory lifecycle. Paper 6.5 supplies stable typed identities, callable-derived tool schemas, declarative skills, lazy disclosure, and host-authorized tool execution. Paper 7 supplies type-aware compact views, scoped lossless backing, selective materialization, and cursors for large results. Paper 8 supplies versioned task state, task-scoped record admission, and durable user/session resolution. We retain these responsibilities and place one orchestration facade above them. This is integration by contract rather than by collapsing distinct policies.

The contributions are:

- a unified `PRARuntime` API over model memory, external resources, typed capabilities, compact result backing, and safe execution;
- a product-facing `PRAAgent` with durable in-memory or local-disk sessions, a validated task DAG, task-scoped context, reusable toolsets, and a resumable terminal interface;
- lazy-by-default registration for Python tools, explicit skills, and OpenAI- or Anthropic-style skill folders, with bounded selection palettes and exact full-view activation;
- session-scoped, hash-verified result backing with type-aware compaction, retrieval-only addresses, selected replay, cursors, and opt-in native PRA routing;
- an explicit physical K/V contract with native grouped-query heads, deduplicated intervals, hard token budgets, transfer accounting, and a byte-bounded LRU;
- independent selection-stage, layer-scope, materialization-lifetime, and residency policies with request-over-model-over-global precedence;
- a standalone G00/G10/G01/G11 gateway, logical JSON wire contract, and E0–E3 engine capability model that makes every downgrade visible;
- stage-level instrumentation and a structured benchmark that separates cold compilation, warm primitives, memory residency, and unsupported paths;
- a thin serving handoff that withholds semantic scores from the scheduler, thereby defining the boundary before deep vLLM integration; and
- a measured eager baseline plus negative compiler and engine gates on the available host.

2 The Systems Gap

2.1 Logical and physical memory are different quantities

Consider L decoder layers. Each reference has T tokens, H_{kv} physical key/value heads, head width D, and b bytes per scalar. Storing complete K/V for the reference requires

$$B_{\text{detail}} = 2LH_{kv}TDb, \tag{1}$$

where the factor two accounts for keys and values. If routing selects S unique tokens, the active payload is

$$B_{\text{active}} = 2L_cH_{kv}SDb, \tag{2}$$

where L_c is the number of memory-consuming layers. The ratio $B_{\text{active}}/B_{\text{detail}}$ is a logical opportunity. It is not peak HBM. Peak HBM also includes model weights, local K/V, routing indices, allocator slack, transfer staging, and temporary packed tensors.

Grouped-query and multi-query attention make the distinction especially important. Warm storage should preserve H_{kv} physical heads. Expanding them to the larger number of query heads before materialization duplicates memory without adding information. The runtime therefore represents every native tensor as

$$K, V \in \mathbb{R}^{B \times H_{kv} \times T \times D} \tag{3}$$

and leaves head expansion to the model-family attention adapter.

2.2 Latency is a sum of visible stages

For one request, PRA overhead is not a single routing timer. We decompose it as

$$t_{\text{PRA}} = t_{\text{lookup}} + t_{\text{graph}} + t_{\text{plan}} + t_{\text{gather}} + t_{\text{transfer}} + t_{\text{pack}} + t_{\text{position}} + t_{\text{metadata}}. \tag{4}$$

Generation and attention follow this preparation. Cold requests also pay source fetch and native encoding; compiled paths pay graph capture and code generation. Reporting only warm attention can therefore reverse the practical conclusion.

3 A Unified Runtime Contract

3.1 One facade, separate authorities

Figure 1 shows the runtime. The facade does not make every component interchangeable in meaning. It gives them one lifecycle and one inspection surface while retaining their ownership boundaries.

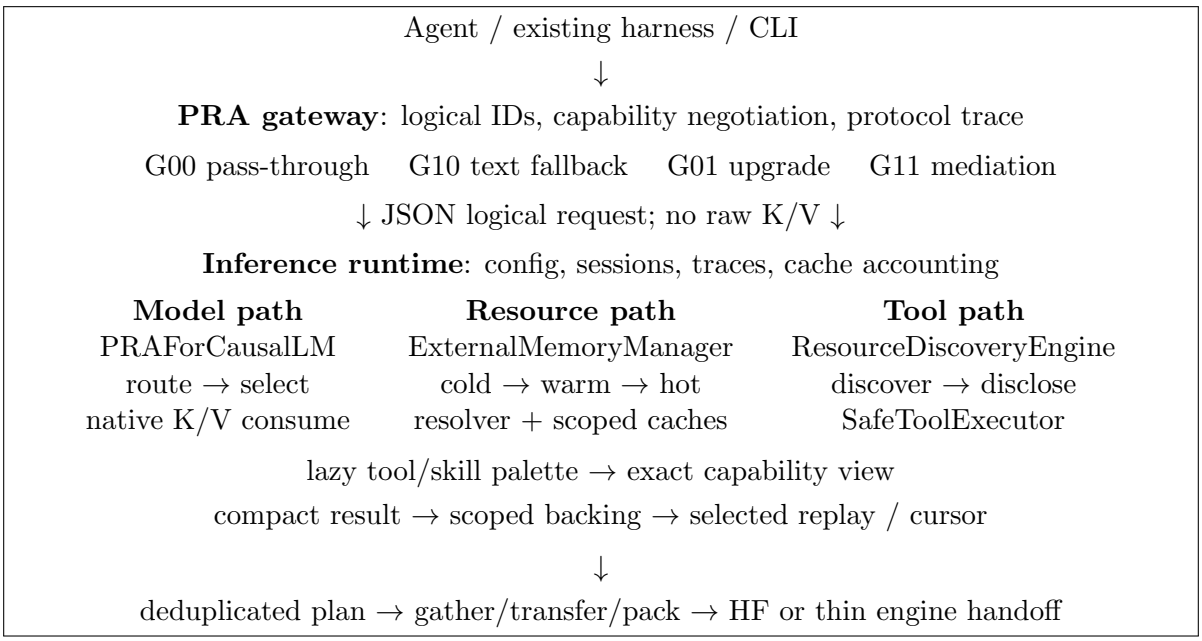

Figure 1: The distributed product boundary. Stable URIs join the paths, but the agent owns task meaning, the gateway owns translation, and the engine owns native K/V. Routing does not authorize source access or tool side effects.

The public configuration has two layers. `PRAConfig` continues to specify semantic behavior: routing layers, chunking, closure mode, and material- ization budget. `PRARuntimeConfig` specifies physical behavior: backend, compilation mode, K/V layout, page size, cache bounds, prefetch policy, and timing synchronization. Both serialize to a versioned JSON artifact. A runtime optimization is valid

only when changing the second layer preserves selected identities and output parity under fixed model semantics.

3.2 A multi-axis execution policy

PRA does not imply one routing frequency. We represent one request with four orthogonal choices. The *selection stage* is request, prefill/completion phase, or token. The *layer scope* is shared logical identities or an independent selection per active layer. The *materialization scope* is request, phase, layer, or token. The *residency policy* retains a payload for that lifetime or releases it after one layer window. Shared identities never mean shared tensors: chunk C_7 resolves to the native K/V independently encoded at each consuming layer.

The global default is request/shared/request/keep. A model default can override individual fields, and a request override takes final precedence. The resolved policy records field-level provenance. Unsupported combinations raise before generation; there is no implicit downgrade. The engine-neutral `PRASelectionPlan` contains stable URI/chunk identities and row-local spans, never tensors. A layer resolver maps that plan to native payloads, and a separate materialization manager applies physical lifetime and residency.

The HF reference path preserves the old high-level request/shared behavior and the old low-level token/per-layer behavior. It adds request/per-layer by capturing all active layers in one memory-disabled prompt pass, routing each once, and freezing each result. It adds token/shared through a request-owned bridge: layers before the designated routing layer receive no current-token PRA memory; that layer selects once, and later layers resolve the same identities to their own K/V. Phase selection is represented but rejected by HF until a cache-correct prefill-to-decode handoff is implemented.

3.3 Stable resources and authenticated sessions

A stable URI is the common identity for text memory, native K/V, tools, and typed observations. Paper 4's `PRASession` retains tenant, user, session, cache scope, admitted resources, and warm/hot handles. Credentials remain opaque callbacks passed only to resolvers; they are absent from prompts, snapshots, and artifacts. Cache presence never grants authorization.

3.4 Durable logical sessions and task scope

An interactive agent needs continuity longer than one model process. The runtime therefore separates logical `AgentSessionState` from physical `PRASession`. Logical state is resolved by user and session identity and contains an ordered typed-record stream, a task descriptor, and an idempotent event log. Physical state owns admitted resources, native K/V, and result-store handles. Closing it releases accelerator and resolver state without deleting the conversation or task graph.

`SessionService` is the persistence boundary. Its in-memory implementation provides thread-safe ephemeral operation. Its local implementation hashes path identities, writes versioned JSON through atomic replacement, and rejects stale optimistic versions. A database can implement the same create, resolve, save, list, and delete contract without changing the model path.

Task events create, link, activate, block, resume, complete, or cancel nodes in a validated DAG. Typed records carry producing-task, consuming-task, relation, and event-sequence provenance. Before ordinary PRA ranking, `TaskScopeSelector` admits the active task, its structural closure, or a bounded adaptive neighborhood. Scope controls which records may compete; record and chunk routing still control which admitted detail is materialized.

3.5 Agent and toolset product surface

`Toolset` couples a callable to its generated schema, namespace, tenant, tags, and declared side-effect class. Bundles compose through the existing `CapabilitySDK`. The built-in local bundle provides file listing, bounded reads and search, exact writes and replacements, Git status, and workspace-rooted command execution. Path operations reject traversal outside the configured workspace. Shell commands remain write-class actions: workspace selection is not a security sandbox.

`PRAAgent` assembles model, capability discovery, lazy skill folders, safe execution, compact results, durable sessions, and task scope. A turn persists the user record, selects task-relevant records, discloses a bounded tool palette, generates, validates at most a configured number of tool rounds, stores compact output with exact-backing provenance, and persists the final assistant record. The terminal adds resumable sessions and slash commands for tasks, context, and tools. Write and destructive calls are denied by default and can receive one host confirmation; disclosure alone never grants authority.

Paper 6.5's tool resources enter through the same runtime but a separate decision path. Discovery can identify a tool. Disclosure can expose its schema. A model can propose a call. Only `SafeToolExecutor`, given a request-scoped `ExecutionAuthorization`, can validate identity, arguments, write authority, and destructive authority. Its result becomes a typed observation resource with producer and call provenance. This separation prevents a high retrieval score from turning into ambient execution privilege.

3.6 Lazy tools and skills

Paper 6.5's latest capability contract is now part of the facade rather than an adjacent experiment. `AgentConfig` accepts ordinary Python callables, explicit declarative `Skill` objects, and a parent directory containing OpenAI- or Anthropic-style skill folders. Callables are inspected into stable, provider-neutral schemas. Folder loaders parse declarative metadata and instructions but never execute bundled scripts.

The default is a two-stage model-side lifecycle. Discovery and Phase A encode only compact selection views under count and token budgets. Once the model or host chooses a stable record ID, `activate_capability` activates the complete tool schema or skill instructions from immutable local backing. It does not rerun semantic discovery. Encoded views may be cached, while active bytes and resident cached bytes remain separate counters. Eager selection or full-view encoding is an explicit policy override.

3.7 Compact typed result backing

A successful tool output need not return as one flat prompt block. Every open `PRASession` now owns an `AdaptiveContextRuntime` with an exact, hash-verified backing store scoped by tenant and session. The runtime infers tabular, log, graph, terminal, or generic structured shape for tool and API results. Type-specific compactors retain schema, counts, statistics, errors, and bounded representative units while lexical, entity, rare-term, schema, and optional summary addresses remain retrieval-only.

The public operations preserve stable record identity: compact inspection, address search, full or selected materialization, and bounded cursor actions. `execute_tool_and_record` joins authorized execution to this record lifecycle while retaining producer, call, and observation provenance. Closing an ephemeral session removes its backing bytes.

Native result routing remains a stronger opt-in. For an isolated Hugging Face model session, three explicit calls define the lifecycle:

`register_result_backing` `route_result_backing` `materialize_routed_result`

They respectively pass exact backing through the production PRA encoder, perform query encoding and bounded native-K/V selection without autoregressive generation, and decode only selected original spans. Ingestion alone never registers exact backing in model memory. Teardown removes all references created by this path. A shared multi-tenant model cache is not claimed safe by this interface.

3.8 User workflow

The supported workflow is deliberately small:

```
agent = AgentConfig(tools=(lookup,), skills_path="./skills")
context = ContextPolicy(record_policies={...})
runtime = PRARuntime.from_pretrained(
    model_id, runtime_config=config,
    agent_config=agent, context_policy=context)
session = runtime.open_session(...)
palette = runtime.activate_capability_candidates(record_ids)
runtime.activate_capability(selected_id)
result = runtime.execute_tool_and_record(..., session=session)
detail = runtime.materialize_result(
    session, result.record.record_id,
    level="selected", selector={"rows": [20, 30]})
trace = runtime.inspect()
```

The same facade still accepts direct references, an `ExternalMemoryManager`, a custom discovery engine, and a safe executor. The executed notebook demonstrates model memory, physical materialization, lazy capabilities, compact results, cursors, and isolated native result routing with an offline Hugging Face model and pure in-memory tools.

4 Physical Materialization

4.1 Planning before copying

Routing produces logical half-open intervals $I_i = (u_i, \ell_i, a_i, b_i)$, naming URI u_i, layer ℓ_i, and token range $[a_i, b_i)$. The planner groups intervals by URI and layer, sorts them, merges overlaps, and then applies a global token budget. If two requested intervals overlap, their common tokens are copied once. The trace reports requested, unique, and dropped token counts, making deduplication and budget truncation auditable.

The portable materializer slices source tensors, transfers selected slices when needed, and packs them by consumption layer. It reports output bytes, transfer bytes, and temporary bytes. It does not infer peak HBM from these values; CUDA profiling records peak allocator state separately.

4.2 Caching and reload amplification

The runtime includes a thread-safe, byte-bounded LRU for opaque warm or hot payloads. Besides hits, misses, and evictions, it records loaded and reused bytes. We define reload amplification as

$$A_{\text{reload}} = \frac{B_{\text{loaded}}}{\max(B_{\text{resident}}, 1)}. \tag{5}$$

An apparently high hit rate can still be costly if the misses repeatedly load large references. Conversely, byte reuse can be high even when object-level hit rate is modest. Both views are needed.

5 Compilation and Engine Boundaries

5.1 Agent, gateway, and engine are independent

The distributed contract has three authorities. The agent owns user-visible task and session semantics, typed records, provenance, and authorization references. The gateway owns transport sessions, logical resource mapping, protocol translation, and explicit capability negotiation. The inference engine owns tensor layout, physical K/V blocks, device placement, attention, and scheduling. The normal wire carries JSON logical identities rather than raw K/V tensors.

The implemented `PRAWireRequest` preserves messages, tenant/session/task IDs, resources, facets, budgets, execution-policy hints, provenance, and required capabilities. `PRAEngineAdapter` reports static capabilities, prepares and closes sessions, and offers generation plus a reserved streaming extension. Four deterministic gateway modes cover both sides of the migration: G00 passes ordinary traffic to an ordinary engine; G10 converts explicitly selected resources to labeled text for a PRA-unaware engine; G01 infers typed resources from system and tool-result boundaries before calling a PRA-aware engine; and G11 preserves an explicit PRA request for a PRA-aware engine. G10 is control-plane PRA with text materialization, not native PRA execution.

Capability negotiation is fail-closed. A request that requires native K/V is rejected by an E0 engine unless it explicitly allows text fallback and selects G10. The trace then names the unsupported capabilities, downgrade, selected logical IDs, gateway mode, correlation ID, and whether native K/V was used. Cross-tenant resources and credential-like request metadata are rejected before translation. Relevance still does not grant source or tool authorization.

We distinguish E0 protocol integration, E1 native PRA attention, E2 a PRA-aware memory runtime, and E3 scheduler-aware batching. The local HF adapter is the E1 semantic reference. The OpenAI-compatible HTTP adapter and standalone `pra gateway serve` process implement E0. SGLang and FreeToken expose compatible HTTP surfaces, so they are reachable at E0; neither has a PRA-native backend in this artifact [6, 2]. Streaming is an interface extension, not an implemented result in this version.

5.2 Compilation ladder

The eager path is the correctness baseline. The first compiled primitive is a paired `index_select` over K and V with dynamic selected-token count. Cold compile time and warm latency are separate rows, and parity is checked against eager outputs. Failure never falls back into an eager row.

Triton, custom CUDA, and fusion follow profiling rather than precede it. The candidate hot paths are interval gather, deduplication/remap, packing, and position preparation. A kernel is justified only if it improves the complete request after launch, transfer, and synchronization costs. This ordering prevents a fast isolated kernel from becoming a misleading end-to-end result.

5.3 Thin vLLM handoff

PagedAttention showed how virtualized K/V blocks improve serving utilization and batching [4]. Deep PRA integration would additionally make logical PRA blocks, semantic residency, and retrieval-aware prefetch visible to the scheduler. That is outside this paper's boundary.

The implemented `VLLMThinBackend` instead emits a `VLLMThinRequest`: prompt, stable selected URIs, materialized token count, and ordinary K/V metadata. Semantic scores do not cross the boundary. An ordinary engine can consume already selected memory without owning retrieval policy. When no executor is installed, the backend produces the handoff for inspection and raises on generation rather than pretending to serve.

6 Relation to Existing Systems

PRA is complementary to several kinds of memory optimization. H2O and SnapKV select or retain influential cache entries inside an active sequence [8, 5]. PRA begins from stable external identities and query-time addressability, so its inactive detail K/V may live outside the accelerator. These methods can eventually share page layouts or eviction mechanisms, but their selection semantics differ.

Prompt Cache and CacheBlend reuse precomputed prompt or retrieved-knowledge state [3, 7]. Their lesson is directly relevant: recomputation and transfer can dominate. PRA adds bounded semantic selection before native K/V consumption. RetrievalAttention similarly joins vector retrieval with long-context attention, reinforcing the need to count index and transfer work rather than only selected attention work.

GQA reduces the number of physical K/V heads [1]. Our runtime preserves this layout through warm storage and materialization. It does not expand K/V to query-head count as an implementation convenience.

7 Experimental Design

7.1 Questions and claim levels

The initial gate asks four narrow questions. First, does the portable selected- K/V implementation preserve exact tensor parity? Second, how large are eager gather and interval-pack costs? Third, how much does CPU residency add for the same selected payload? Fourth, can the local toolchain execute the compiled path? Quality and routing selection remain frozen.

Claims are labeled by evidence. *Measured* means executed on the recorded host. *Inherited* means read from a checked-in earlier experiment with its own protocol. *Contract only* means code exists but no external engine ran. *Architectural* means only an interface comparison is made.

7.2 Portable primitive protocol

The new benchmark uses PyTorch 2.12.1 with CUDA 12.6 on Windows 10, Python 3.10.11, and an NVIDIA GeForce GTX 950M with compute capability 5.0. Each synthetic tensor has four physical K/V heads, width 64, and 8,192 candidate tokens in float32. Selection admits 512 tokens (6.25%). Batch sizes are one and four. CPU rows use five warmups and 30 timed repetitions; CUDA rows use ten warmups and 50 repetitions. Every CUDA timing synchronizes before and after the operation.

Indexed gather uses evenly distributed selected token IDs. Interval packing uses eight intervals under the same 512-token budget. The hierarchy comparison holds intervals and output bytes fixed while moving sources among HBM, pinned CPU, and ordinary CPU. Compiler errors are artifact rows. The benchmark is a mechanism profile, not TTFT or throughput for a language model.

The layout sweep holds 8,192 total source tokens across four references and two layers. It packs 32-token pages in layer-major, reference-major, chunk-major, or block-major order, then restores an identical 512-token logical selection. Store construction is a cold cost; reported layout gather latency follows the same warmup and repetition protocol as the other device rows.

7.3 Inherited profiles

Two earlier artifacts answer adjacent questions. The Paper 2 public-API demo ran frozen Qwen3-0.6B over two QASPER and two HotpotQA examples. It records reference ingest, query encoding, routing, generation, K/V fractions, transfers, and peak GPU allocation. The Paper 4 lifecycle fixture ran eight deterministic resources through authenticated cold, warm, and hot states. Its answer proxy

is required-document availability, not language quality. We do not pool these measurements with the new microbenchmark or compute cross-host speedups.

7.4 Execution-policy mechanism profile

We instantiate the same random-weight, three-layer tiny Llama configuration for five seeds and ingest two stable synthetic references. Every condition uses three active PRA layers, deterministic decoding, and at most three generated tokens. We compare request/shared at first and last routing layers, request/per-layer, token/shared at first and last routing layers, and token/per-layer. The profile records routing and selection epochs, temporal and layer Jaccard, selected-identity agreement, and generation wall time. It tests execution semantics and relative routing work, not language quality or production latency. EOS can terminate before three tokens, so token-level operation counts are averaged over the actual forwards.

8 Results

8.1 Selection sharing removes repeated routing work

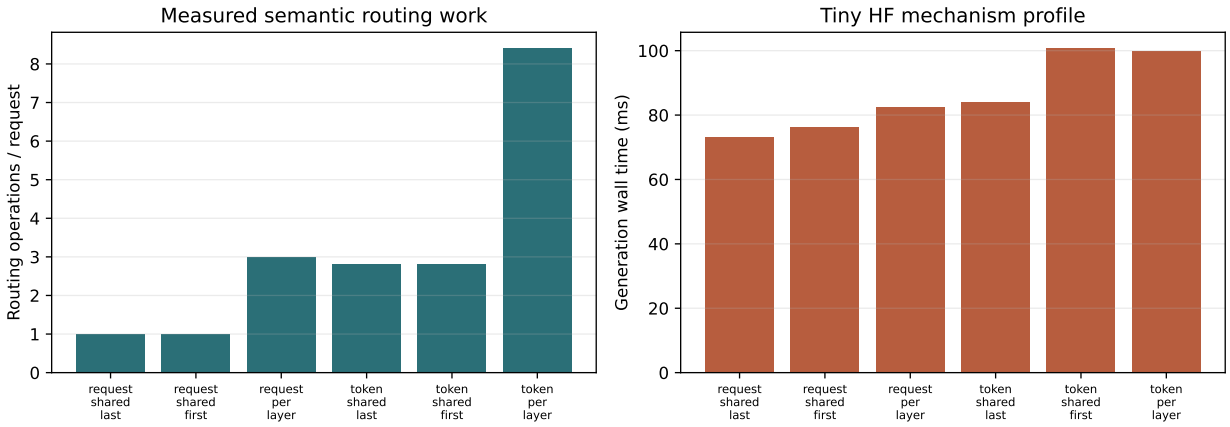

Figure 2: Five-seed tiny-HF mechanism profile. Routing operations validate the execution contract; wall time is host- and model-specific and is not a serving speed claim.

Table 1: Implemented policy points in the five-seed mechanism profile. Times are means over random-weight tiny models and do not measure answer quality.

Condition	Routing operations	Generation ms
Request/shared, last	1.0	73.1
Request/per-layer	3.0	82.6
Token/shared, first	2.8	100.7
Token/per-layer	8.4	100.0

Request/per-layer performs exactly three routing decisions, but amortizes them over generation. Token/shared and token/per-layer observe the same average number of autoregressive forwards; sharing reduces routing work by 3.0×, from 8.4 to 2.8 operations. Tiny-model wall times are effectively tied for the two token modes, which means Python and model overhead dominate at this scale. The

routing-layer sweep also changed selected identities, confirming that routing layer is a semantic parameter rather than a scheduling-only knob. The stable selections within these short completions yield temporal Jaccard one; longer trained-model studies are needed before making a churn or quality claim.

8.2 Portable eager primitives

Table 2: Median warm latency in milliseconds. All measured outputs match the eager reference exactly. Each output contains 512 selected tokens per request.

Device	Primitive	Batch 1	Batch 4
CPU	indexed gather	0.133	0.835
CPU	interval pack	0.228	0.938
CUDA	indexed gather	0.253	0.478
CUDA	interval pack	0.415	0.613

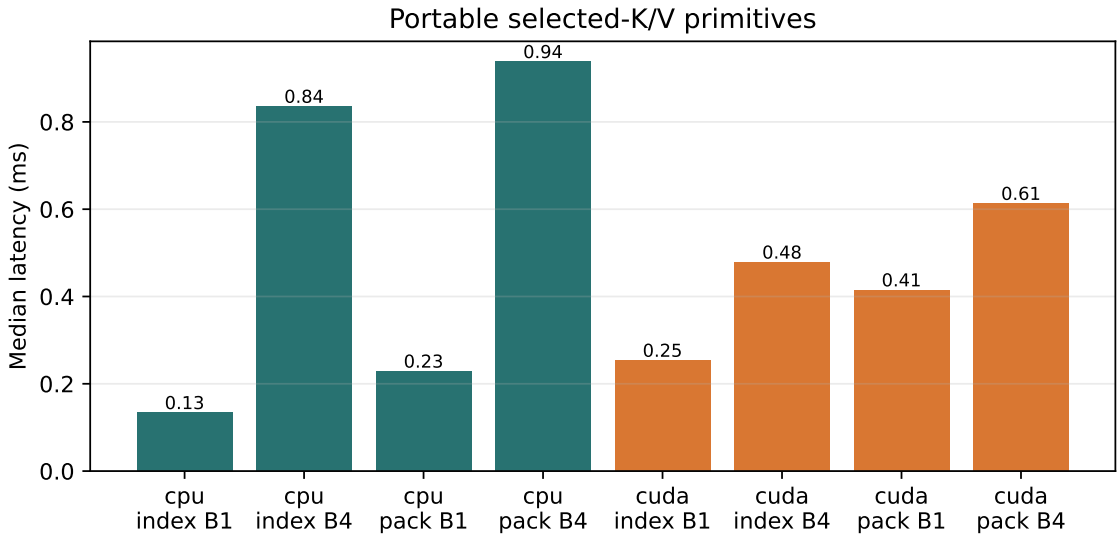

Figure 3: Portable primitive latency. This older GPU does not make small batch-one gathers faster than the CPU, but CUDA scales better at batch four.

The indexed CUDA path is 0.253 ms at batch one. The eight-interval pack is 0.415 ms because it launches and concatenates multiple slices. At batch four, CUDA indexed throughput increases to roughly 4.28 million selected tokens/s, whereas the CPU reaches about 2.45 million million. These values identify packing and fragmentation as optimization targets; they do not include routing or attention.

8.3 Residency dominates the selected payload

Table 3: Median CUDA interval materialization latency for an identical selected payload. Output size is 1 MiB at batch one and 4 MiB at batch four.

Source residency	Batch 1 (ms)	Batch 4 (ms)
HBM resident	0.424	3.371
Pinned CPU → CUDA	1.592	6.655
Ordinary CPU → CUDA	2.157	8.660

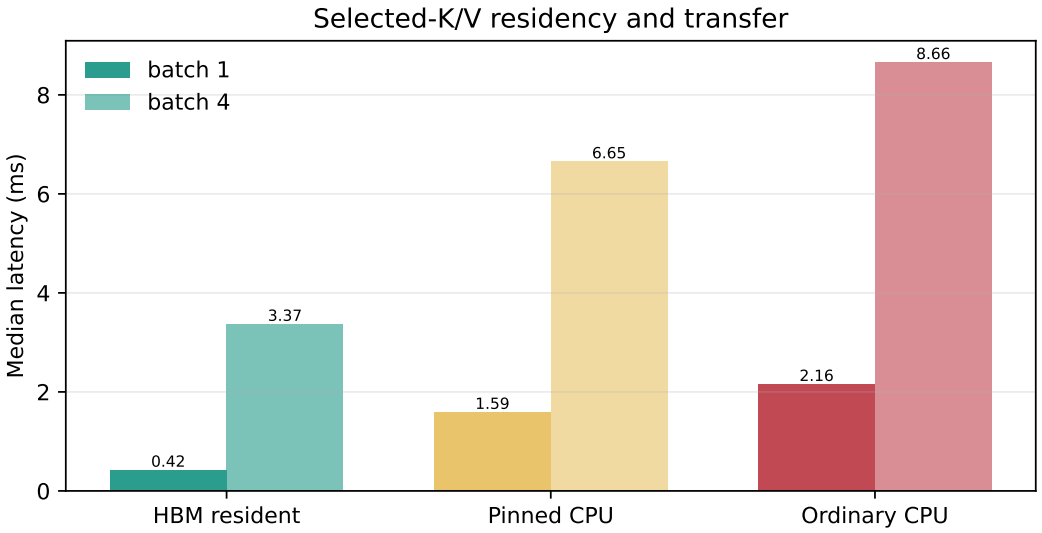

Figure 4: The same logical selection has different physical cost depending on residency. Pinned transfer helps, but does not erase the hierarchy boundary.

At batch one, pinned transfer is 3.75 times the HBM-resident cost, and ordinary CPU transfer is 5.08 times that cost. This is the central result of the initial systems gate. A configuration that moves inactive K/V off accelerator reduces residency, but request latency then depends on locality, cache reuse, and transfer overlap. Any future headline must report both memory and latency.

8.4 Physical layout is secondary to fragmented remapping

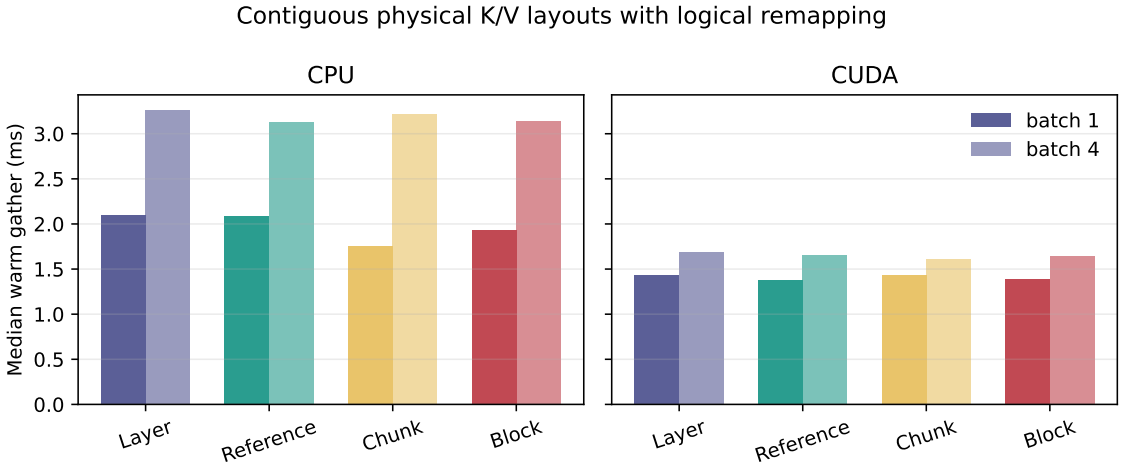

Figure 5: Warm materialization from four physically distinct contiguous stores. Every layout restores the same logical per-layer K/V before attention.

Layer-major, reference-major, chunk-major, and block-major stores place the same four-reference, two-layer native K/V in different contiguous orders. A compact placement index maps URI, layer, and logical page offsets back to physical ranges. All outputs match the unpacked reference exactly. On CUDA, the slowest and fastest medians differ by only 3.8% at batch one and 5.0% at batch four. The nominal fastest layouts are reference-major and chunk-major, respectively, but those small differences do not justify a universal preference on this synthetic cohort.

More importantly, all layout-remap paths take roughly 1.4–1.7 ms on CUDA, several times the single-source interval pack. They reconstruct selected logical ranges across Python placement records and page fragments. The next optimization target is therefore indexed vectorized remapping and fewer launches, not a claim that one physical ordering has won.

8.5 Compilation is a negative capability gate

The `torch.compile` API is present, but neither local compiled path is valid. CPU Inductor cannot find the Microsoft C/C++ compiler. CUDA compilation rejects compute capability 5.0 because the Triton backend requires capability 7.0 or newer. No warm compiled latency is reported and no eager result is relabeled as compiled. Triton and custom CUDA therefore remain untested.

8.6 Earlier profiles locate larger costs

The new primitive measurements explain the cost of moving an already selected payload. Earlier end-to-end artifacts provide the complementary scale: how much time was spent before and after materialization when the public model API and authenticated resource lifecycle were exercised.

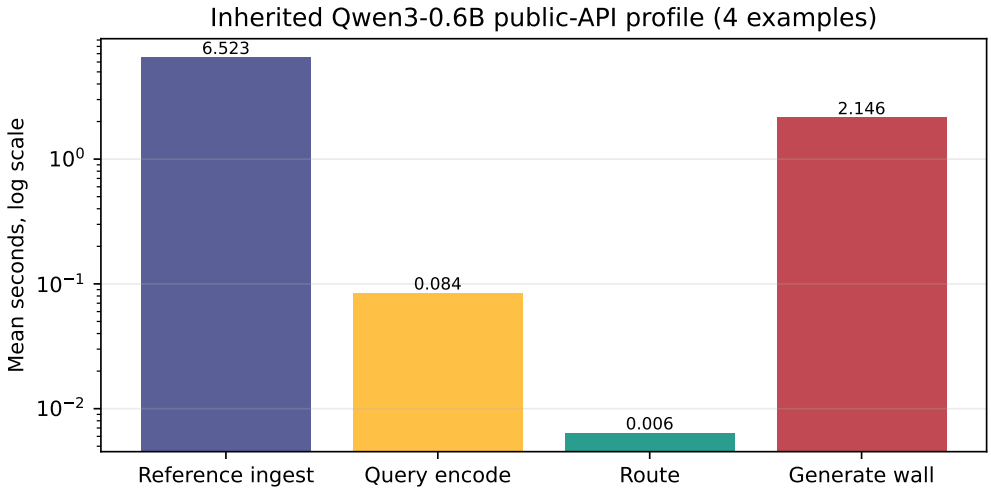

Figure 6: Inherited Paper 2 Qwen3-0.6B profile over four examples. The log scale shows that routing time is small relative to native reference encoding and generation on that protocol.

The inherited profile reports 6.40 ms mean routing time, 6.52 s mean reference ingest, and 2.15 s mean routed generation wall time. Mean evidence recall is 0.667 and materialized detail K/V is 0.066 of the candidate total. This supports optimizing cache reuse and encoding amortization before replacing the router with a custom kernel. The cohort contains only four examples, so it does not establish a general workload distribution.

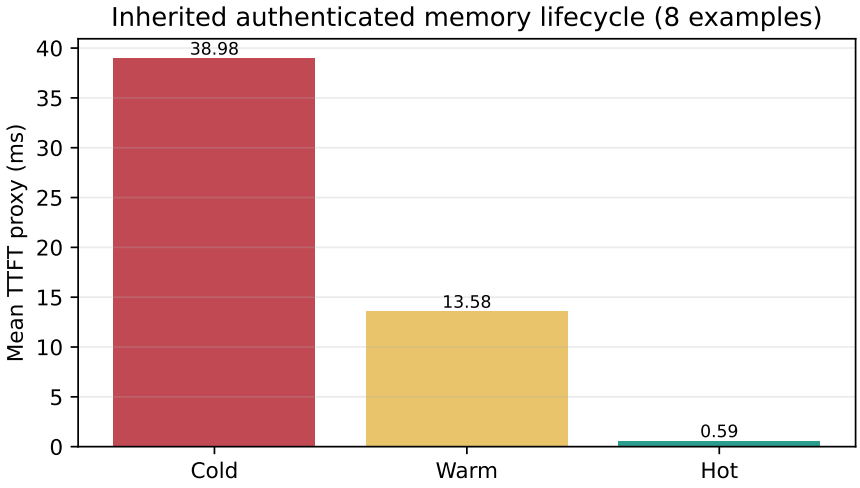

Figure 7: Inherited Paper 4 lifecycle proxy over eight examples. Cold includes fetch and native encoding; warm reuses native encoding; hot reuses selected K/V.

The lifecycle TTFT proxy falls from 38.98 ms cold to 13.58 ms warm and 0.59 ms hot. Required-document recall is 0.875. The result validates state transitions and shows the size of the reuse opportunity, but it is not an LM answer-quality or production-serving measurement.

9 Compatibility and Product Boundary

Table 4: Implementation status on the measured branch and host.

Runtime	Status	Boundary
HF/PyTorch eager	measured	Native PRA model API and portable K/V primitives
Standalone gateway	contract tested	G00/G10/G01/G11 JSON mediation; non-streaming
OpenAI-compatible HTTP	E0 implemented	Pass-through and explicit text fallback
`torch.compile`	blocked on host	API implemented; compiler/GPU gates recorded
Triton/custom CUDA	not run	Requires supported GPU and profiling evidence
vLLM thin	contract only	Scheduler-unaware identity and K/V handoff
SGLang/FreeToken	E0 feasible, not run	Compatible HTTP; no native PRA backend
TensorRT-LLM/MLX	architectural only	Not installed on measured host

The product surface includes installable package metadata, Hugging Face loading, runtime config serialization, direct and external reference registration, session isolation, callable and folder-backed capability registration, lazy view activation, typed resource discovery, safe execution, compact result ingestion, selected replay, address search, cursors, opt-in native result routing, generation, inspection, benchmark commands, and two executed notebooks. The CLI exposes `runtime init`, `inspect`, `capabilities`, `benchmark`, `prepare-vllm`, and a persistent `agent chat` terminal. The `pra gateway serve` and `pra-hf gateway serve` commands launch the same independently testable gateway with health, capability, logical-request, and OpenAI-compatible endpoints. A deployment can therefore discover unsupported paths before serving.

10 Limitations and Next Gates

The current evidence is intentionally incomplete. The GPU is old, the benchmark is synthetic, and no full serving engine is installed. We have not measured continuous batching, request-level p95/p99, asynchronous transfer overlap, a production page allocator, prefetch, multi-session contention, or fused attention preparation. The LRU fixture validates byte accounting but is not a production cache policy study. The inherited language-model cohort is too small for quality claims. The new capability and compact-result workflows and Paper 8 task/session integration are executed contract tests, not a new latency or answer-quality experiment. The shell is intentionally single-agent; subagents and learned long-term memory remain scoped to Papers 9 and 10. Native result routing currently requires an isolated model session; safe concurrent multi-tenant partitioning of model-resident result K/V remains an engine-level gate. The multi-axis HF bridge serializes high-level generation on one wrapped model to isolate temporary adapter state. It is a correctness reference, not a continuous-batching implementation. Phase selection, request-resident physical payload reuse across changing direct-token budgets, streaming gateway proxying, and E2/E3 engine integration remain open. The G01 parser recognizes explicit system, tool, and attachment boundaries; it does not claim semantic recovery from arbitrary opaque traffic.

The next experiment should move to a compute-capability 8.0 or newer GPU with a pinned software stack. It should first reproduce eager parity, then measure `torch.compile` cold and warm paths. Only measured hot spots should move to Triton. Layout and hierarchy sweeps should cross contiguous

versus indexed selection, whole chunks versus intervals, GQA head count, batch heterogeneity, and warm-cache locality. End-to-end runs should report TTFT, inter-token latency, throughput, peak HBM, off-device bytes, temporary bytes, transfers, and reload amplification together.

The next policy experiment should use a trained model and retrieval workload, hold selected-token budgets fixed, and compare answer quality, TTFT, inter-token latency, selection churn, and routing-layer disagreement. Phase/shared requires a cache-correct prefill handoff before it enters that comparison.

The first deeper engine integration should remain thin: external routing, selected native-K/V materialization, then ordinary vLLM execution. Disabled-PRA parity, native positions, masks, GQA, and batching must pass before speed comparisons. If useful behavior requires semantic residency, retrieval-aware scheduling, or cross-request PRA block sharing, that work belongs to the deep engine paper.

11 Conclusion

PRA's selected-memory fraction is an algorithmic fact, not a systems result. This paper turns that distinction into code and measurement. One runtime now connects the public model API, authenticated memory lifecycle, typed resources, lazy tool and skill disclosure, safe execution, compact result backing, selective replay, durable tasks and sessions, physical K/V planning, caching, profiling, and backend handoff. Its policy lattice makes routing frequency and physical lifetime inspectable, while the gateway permits ordinary and PRA-aware agents and engines to coexist without sending native tensors over the logical wire. The eager primitives are exact and inexpensive in isolation, but off-device transfer multiplies their cost on the measured host. Compilation and optional engines remain negative or unmeasured gates. The practical conclusion is conservative: cache locality, physical residency, and context lifecycle deserve attention before kernel novelty, and every future speed claim must match semantics while accounting for the whole request.

A Reimplementation Guide

The stable public entry points live in `src/pra_hf/runtime.py`. A reimplementation needs five small contracts. First, represent warm K/V as equal-shaped tensors $[B, H_{kv}, T, D]$. Second, express selections as stable URI, layer, and half-open token intervals. Third, merge overlaps before applying a hard global token budget. Fourth, pack selected tensors by consumption layer without expanding grouped-query heads. Fifth, emit stage times and physical bytes independently of routing-quality metrics.

The model adapter only needs `add_reference`, `generate`, and `inspect`. The thin serving request contains selected identities and materialized-token metadata, never semantic scores. Tool execution receives a separate authorization object after discovery. These boundaries allow another runtime to reproduce the interface without adopting the Python implementation.

A.1 Execution and deployment code map

Line references identify the artifact version accompanying this paper. `src/pra_torch/execution.py:61--107` defines the four policy axes and their validation; lines 358–425 define tensor-free identities and row-local plans; lines 428–504 own request epochs, churn, and trace summaries; and lines 507–632 separate selection reuse from opaque materialization lifetimes. The important invariant is the type boundary: a selection controller returns URI/chunk spans, while only an engine resolver may return tensors or block handles.

`src/pra_hf/hf_execution.py:49--174` is the family-independent HF bridge. At the routing layer it turns the current hidden representation into a logical plan. For shared scope it remaps the source-

layer hits through `map_chunk_identities_to_layers`; each destination therefore receives its own native grouped-query K/V. The public request lock and precedence entry point are in `src/pra_hf/model.py:704--889`. Qwen, Llama, and Gemma call the same bridge from their thin attention adapter.

For a distributed implementation, begin with `src/pra_hf/deployment.py:100--173`: reproduce that JSON request without adding tensor fields. At lines 175–182, expose capabilities, session setup, generation, streaming, and close as separate methods. Then reproduce deterministic mediation in `src/pra_hf/gateway.py:24--141`: negotiate first, transform according to G00–G11 second, and append component-local trace events last. The HTTP process starts at line 183; the CLI binding is `src/pra_hf/gateway_cli.py:35--59`. This order keeps an unsupported native request from being mistaken for successful text fallback.

B Artifact Map

Artifact	Role
`src/pra_hf/runtime.py`	Unified facade, plans, materializer, cache, profiler, HF adapter, thin vLLM contract
`src/pra_torch/execution.py`	Engine-neutral policy axes, logical plans, precedence, lifecycle, and analytical routing count
`src/pra_hf/hf_execution.py`	Request-owned token/shared and token/per-layer HF execution bridge
`src/pra_hf/deployment.py`, `src/pra_hf/gateway.py`	JSON wire types, engine capabilities, E0/E1 adapters, G00–G11 mediation, and HTTP process
`experiments/paper4_5_runtime/run_execution_policy_profile.py`	Five-seed multi-axis mechanism profile and trade-off figure
`src/pra_hf/runtime_benchmark.py`	Structured eager/compiled and hierarchy microbenchmark
`src/pra_hf/agent_resources.py`	Typed resource identities and discovery
`src/pra_hf/agent_execution.py`	Schema validation and independent host authorization
`src/pra_hf/agent.py`	Task-aware turn orchestration and persistent agent facade
`src/pra_hf/session_service.py`	In-memory and atomic local logical session persistence
`src/pra_hf/task_context.py`, `src/pra_hf/task_scope.py`	Versioned task DAG, provenance, scope admission, and working-set lifecycle
`src/pra_hf/toolsets.py`, `src/pra_hf/tui.py`	Reusable tools and the resumable coding-agent terminal
`src/pra_hf/capability_sdk.py`	Callable and skill-folder ingestion, bounded palettes, and exact capability activation
`src/pra_hf/typed_context.py`	Type-aware compact views and retrieval addresses for large results
`src/pra_hf/adaptive_context_runtime.py`	Scoped backing, materialization, transport accounting, and cursors
`src/pra_hf/progressive_context.py`	Stable record views and optional production PRA routing over exact backing
`src/pra_hf/external_memory.py`	Resolver-neutral cold/warm/hot lifecycle and scoped caches
`pra-hf-demo/pra_runtime_productization.ipynb`	Executed user workflow

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints.

In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[2] FlashML.org. Freetoken quickstart. `https://github.com/FlashML-org/FreeToken/blob/main/docs/quickstart.md`, 2026. Accessed 2026-08-27.

[3] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[5] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[6] SGLang Project. Sglang quickstart and attention backend documentation. `https://github.com/sgl-project/sglang/tree/main/docs`, 2026. Accessed 2026-08-27.

[7] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[8] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: `https://openreview.net/forum?id=RkRrPp7GKO`.